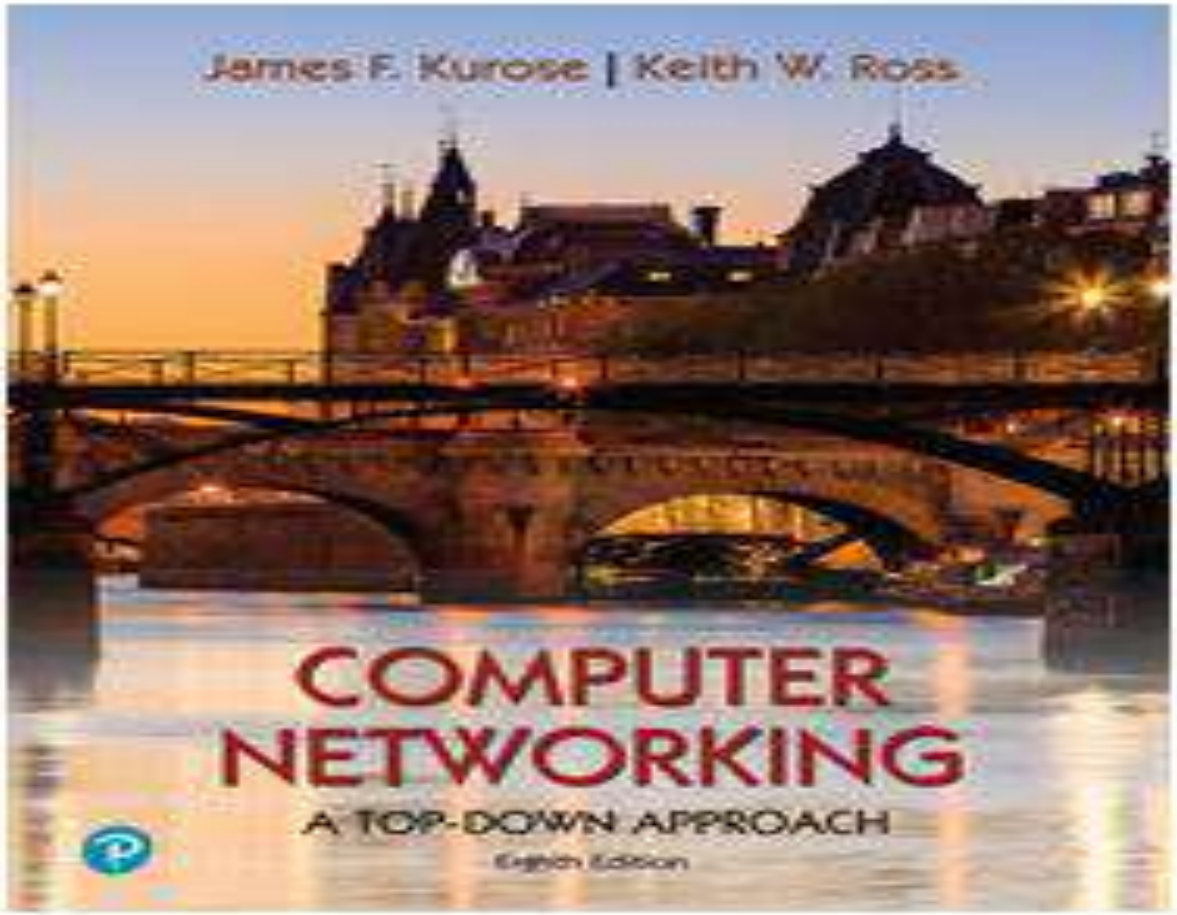


Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Chapter 2 Application Layer

Since the Internet's inception, numerous useful and entertaining applications have indeed been created.
These applications have been the driving force behind the Internet's success, motivating people in homes, schools, governments, and businesses to make the Internet an integral part of their daily activities.

Internet applications include the classic text-based applications that became popular in the 1970s and 1980s: text e-mail, remote access to computers, file transfers, and newsgroups. They include *the* killer application of the mid-1990s, the World Wide Web, encompassing Web surfing, search, and electronic commerce.

Since the beginning of new millennium, new and highly compelling applications continue to emerge, including voice over IP and video conferencing such as Skype, Facetime, and Google Hangouts; user generated video such as YouTube and movies on demand such as Netflix; and multiplayer online games such as Second Life and World of Warcraft.

During this same period, we have seen the emergence of a new generation of social networking applications—such as Facebook, Instagram, and Twitter—which have created human networks on top of the Internet's network or routers and communication links.

And most recently, along with the arrival of the smartphone and the ubiquity of 4G/5G wireless Internet access, there has been a profusion of location based mobile apps, including popular check-in, dating, and road-traffic forecasting apps (such as Yelp, Tinder, and Waz), mobile payment apps (such as WeChat and Apple Pay) and messaging apps (such as WeChat and WhatsApp).
Clearly, there has been no slowing down of new and exciting Internet applications.

Perhaps some of the readers of this text will create the next generation of killer Internet applications!

In this chapter, we study the conceptual and implementation aspects of network applications.
We begin by defining key application-layer concepts, including network services required by applications, clients and servers, processes, and transport-layer interfaces.

We examine several network applications in detail, including the Web, e-mail, DNS, peer-to-peer (P2P) file distribution, and video streaming.
We then cover network application development, over both TCP and UDP.
In particular, we study the socket interface

2.1 Principles of Network Applications

At the core of network application development is writing programs that run on different end systems and communicate with each other over the network.

For example, in the Web application there are two distinct programs that communicate with each other: the browser program running in the user's host (desktop, laptop, tablet, smartphone, and so on); and the Web server program running in the Web server host.

As another example, in a Video on Demand application such as Netflix, there is a Netflix-provided program running on the user's smartphone, tablet, or computer; and a Netflix server program running on the Netflix server host.

Servers often (but certainly not always) are housed in a data center,
Thus, when developing your new application, you need to write software that will run on multiple end systems.

This software could be written, for example, in C, Java, or Python.
Importantly, you do not need to write software that runs on network-core devices, such as routers or link-layer switches.
Even if you wanted to write application software for these network-core devices, you wouldn't be able to do so.

This basic design—namely, confining application software to the end systems, has facilitated the rapid development and deployment of a vast array of network applications.

2.1.1 Network Application Architectures

Keep in mind that an application's architecture is distinctly different from the network architecture (e.g., the five-layer Internet architecture discussed in Chapter 1).

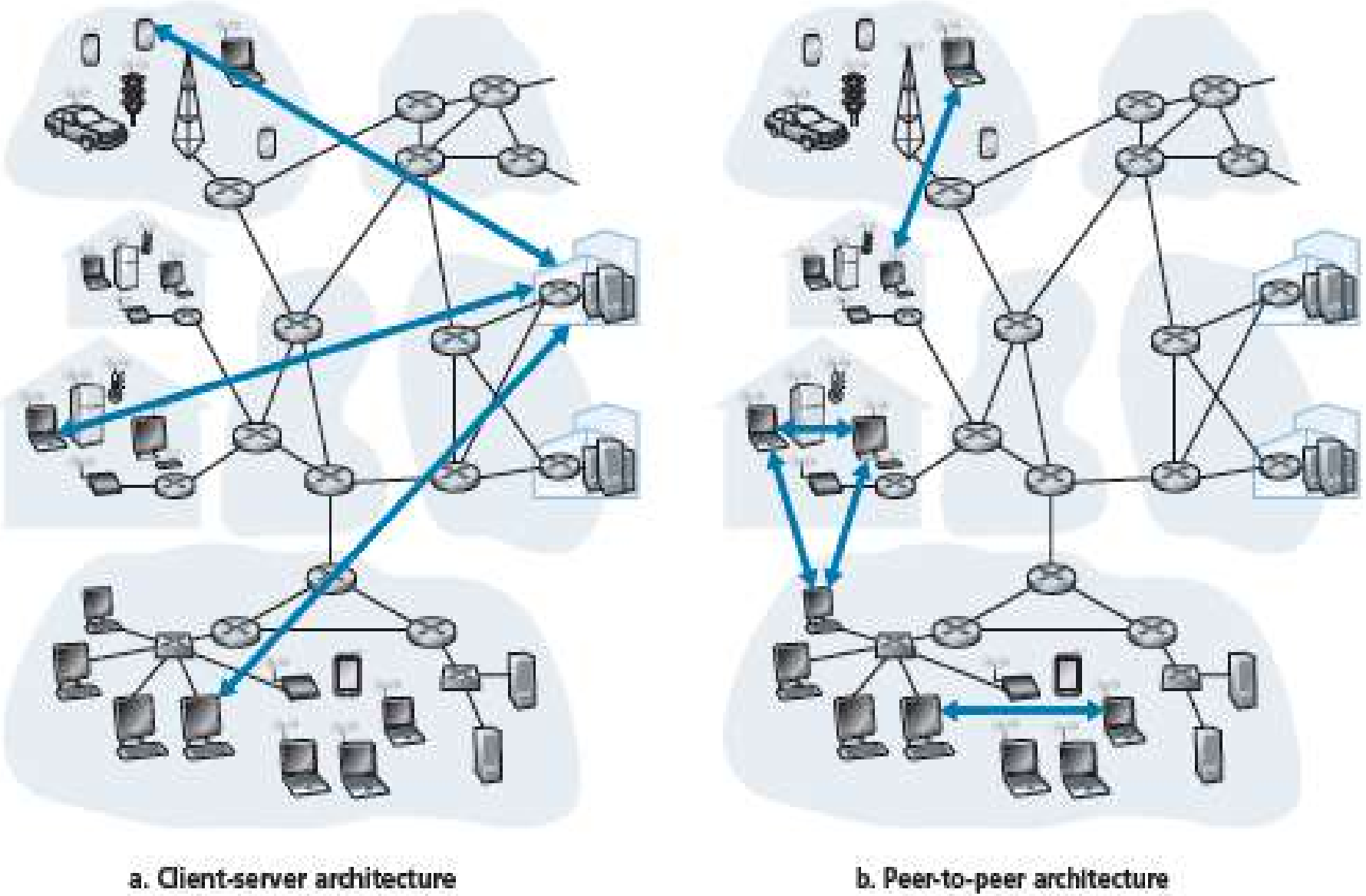
From the application developer's perspective, the network architecture is fixed and provides a specific set of services to applications.

The **application architecture**, on the other hand, is designed by the application developer and dictates how the application is structured over the various end systems.
In choosing the application architecture, an application developer will likely draw on one of the two predominant architectural paradigms used in modern network applications: the client-server architecture or the peer-to-peer (P2P) architecture.

In a **client-server architecture**, there is an always-on host, called the *server*, which services requests from many other hosts, called *clients*.
A classic example is the Web application for which an always-on Web server services requests from browsers running on client hosts.
When a Web server receives a request for an object from a client host, it responds by sending the requested object to the client host.

Note that with the client-server architecture, clients do not directly communicate with each other; for example, in the Web application, two browsers do not directly communicate.

Another characteristic of the client-server architecture is that the server has a fixed, well-known address, called an IP address (which we'll discuss soon).
Because the server has a fixed, well-known address, and because the server is always on, a client can always contact the server by sending a packet to the server's IP address.
Some of the better-known applications with a client-server architecture include the Web, FTP, Telnet, and e-mail.
The client-server architecture is shown in Figure 2.2(a).



a. Client-server architecture

b. Peer-to-peer architecture

Figure 2.2 ♦ (a) Client-server architecture; (b) P2P architecture

Often in a client-server application, a single-server host is incapable of keeping up with all the requests from clients.

For example, a popular social-networking site can quickly become overwhelmed if it has only one server handling all of its requests.

For this reason, a **data center**, housing a large number of hosts, is often used to create a powerful virtual server.
The most popular Internet services—such as search engines (e.g., Google, Bing, Baidu), Internet commerce (e.g., Amazon, eBay, Alibaba), Web-based e-mail (e.g., Gmail and Yahoo Mail), social media (e.g., Facebook, Instagram, Twitter, and WeChat)—run in one or more data centers.

In a **P2P architecture**, there is minimal (or no) reliance on dedicated servers in data centers. Instead the application exploits direct communication between pairs of intermittently connected hosts, called *peers*.
The peers are not owned by the service provider, but are instead desktops and laptops controlled by users, with most of the peers residing in homes, universities, and offices. Because the peers communicate without passing through a dedicated server, the architecture is called peer-to-peer.

An example of a popular P2P application is the file-sharing application BitTorrent.

One of the most compelling features of P2P architectures is their **self-scalability**.
For example, in a P2P file-sharing application, although each peer generates workload by requesting files, each peer also adds service capacity to the system by distributing files to other peers.

P2P architectures are also cost effective, since they normally don't require significant server infrastructure and server bandwidth (in contrast with clients-server designs with datacenters).

However, P2P applications face challenges of security, performance, and reliability due to their highly decentralized structure.

2.1.2 Processes Communicating

Before building your network application, you also need a basic understanding of how the programs, running in multiple end systems, communicate with each other.

In the jargon of operating systems, it is not actually programs but **processes** that communicate.

A process can be thought of as a program that is running within an end system.
When processes are running on the same end system, they can communicate with each other with interprocess communication, using rules that are governed by the end system's operating system.
But in this book, we are not particularly interested in how processes in the same host communicate, but instead in how processes running on *different* hosts (with potentially different operating systems) communicate.

Processes on two different end systems communicate with each other by exchanging **messages** across the computer network.
A sending process creates and sends messages into the network; a receiving process receives these messages and possibly responds by sending messages back.

processes communicating with each other reside in the application layer of the five-layer protocol stack.

Client and Server Processes

A network application consists of pairs of processes that send messages to each other over a network.
For example, in the Web application a client browser process exchanges messages with a Web server process.

In a P2P file-sharing system, a file is transferred from a process in one peer to a process in another peer.

For each pair of communicating processes, we typically label one of the two processes as the **client** and the other process as the **server**.

With the Web, a browser is a client process and a Web server is a server process.

With P2P file sharing, the peer that is downloading the file is labeled as the client, and the peer that is uploading the file is labeled as the server.
You may have observed that in some applications, such as in P2P file sharing, a process can be both a client and a server.
Indeed, a process in a P2P file-sharing system can both upload and download files.
Nevertheless, in the context of any given communication session between a pair of processes, we can still label one process as the client and the other process as the server.

We define the client and server processes as follows:

In the context of a communication session between a pair of processes, the process that initiates the communication (that is, initially contacts the other process at the beginning of the session) is labeled as the client*.*
The process that waits to be contacted to begin the session is the server*.*

In the Web, a browser process initializes contact with a Web server process; hence the browser process is the client and the Web server process is the server.

In P2P file sharing, when Peer A asks Peer B to send a specific file, Peer A is the client and Peer B is the server in the context of this specific communication session.
When there's no confusion, we'll sometimes also use the terminology "client side and server side of an application."

The Interface Between the Process and the Computer Network

As noted above, most applications consist of pairs of communicating processes, with the two processes in each pair sending messages to each other.
Any message sent from one process to another must go through the underlying network.

A process sends messages into, and receives messages from, the network through a software interface called a **socket**.

Let's consider an analogy to help us understand processes and sockets.
A process is analogous to a house and its socket is analogous to its door.
When a process wants to send a message to another process on another host, it shoves the message out its door (socket).

This sending process assumes that there is a transportation infrastructure on the other side of its door that will transport the message to the door of the destination process..

Once the message arrives at the destination host, the message passes through the receiving process's door (socket), and the receiving process then acts on the message.

Figure 2.3 illustrates socket communication between two processes that communicate over the Internet.
(Figure 2.3 assumes that the underlying transport protocol used by the processes is the Internet's TCP protocol.)

As shown in this figure, a socket is the interface between the application layer and the transport layer within a host.

It is also referred to as the **Application Programming Interface (API)** between the application and the network, since the socket is the programming interface with which network applications are built.

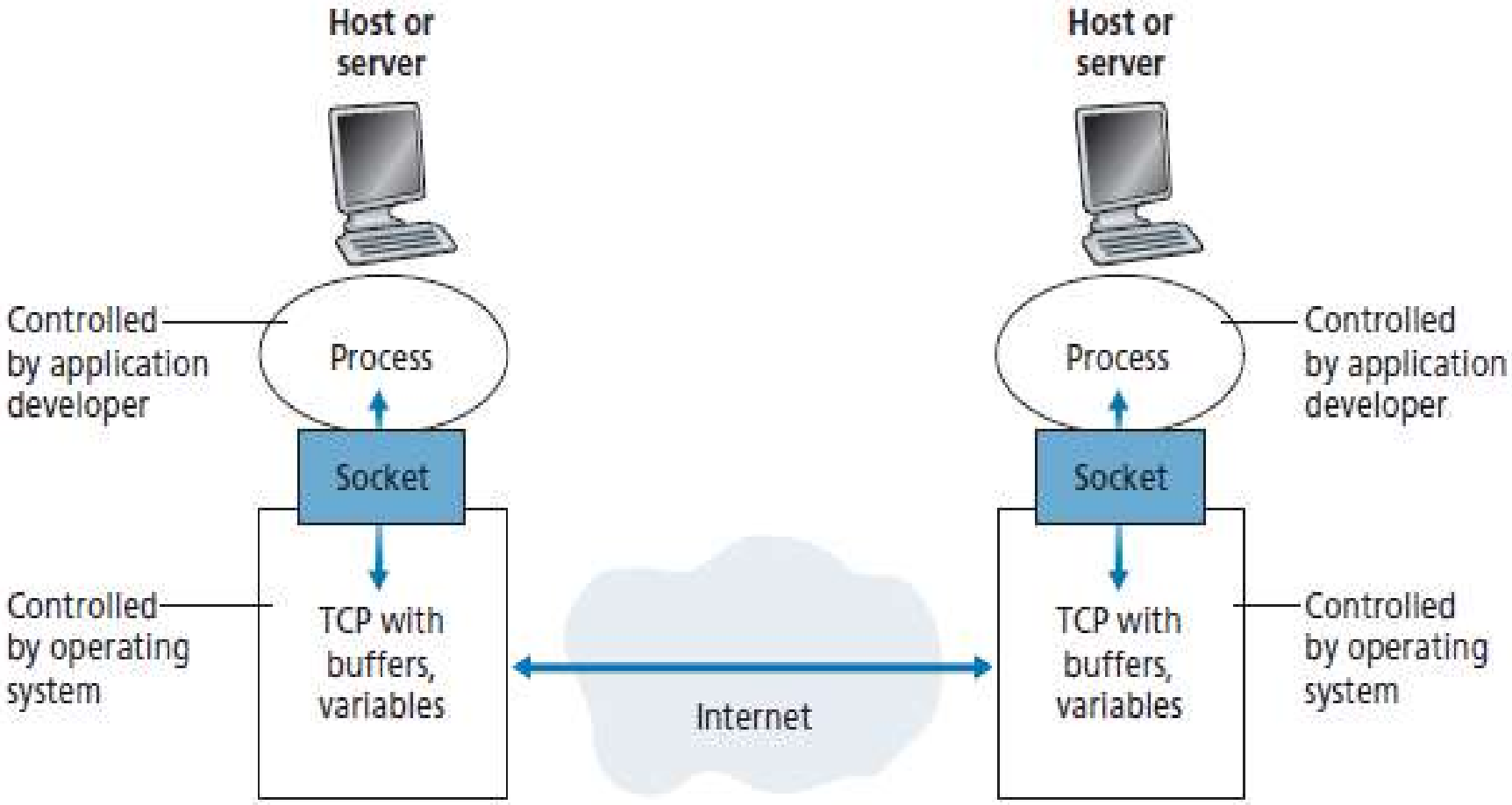


Figure 2.3 ♦ Application processes, sockets, and underlying transport protocol

The application developer has control of everything on the application-layer side of the socket but has little control of the transport-layer side of the socket.

The only control that the application developer has on the transport-layer side is
(1) the choice of transport protocol and
(2) perhaps the ability to fix a few transport-layer parameters such as maximum buffer and maximum segment sizes (to be covered in Chapter 3).

Once the application developer chooses a transport protocol (if a choice is available), the application is built using the transport-layer services provided by that protocol.

Addressing Processes

In order to send postal mail to a particular destination, the destination needs to have an address. Similarly, in order for a process running on one host to send packets to a process running on another host, the receiving process needs to have an address.

To identify the receiving process, two pieces of information need to be specified:
(1) the address of the host and
(2) an identifier that specifies the receiving process in the destination host.

In the Internet, the host is identified by its **IP address**.
We'll discuss IP addresses in great detail in Chapter 4.
For now, all we need to know is that an IP address is a 32-bit quantity that we can think of as uniquely identifying the host.

In addition to knowing the address of the host to which a message is destined, the sending process must also identify the receiving process (more specifically, the receiving socket) running in the host. This information is needed because in general a host could be running many network applications.
A destination **port number** serves this purpose.

Popular applications have been assigned specific port numbers.
For example, a Web server is identified by port number 80.
A mail server process (using the SMTP protocol) is identified by port number 25.

A list of well-known port numbers for all Internet standard protocols can be found at www.iana.org.

2.1.3 Transport Services Available to Applications

Recall that a socket is the interface between the application process and the transport-layer protocol.
The application at the sending side pushes messages through the socket.
At the other side of the socket, the transport-layer protocol has the responsibility of getting the messages to the socket of the receiving process.

Many networks, including the Internet, provide more than one transport-layer protocol.
When you develop an application, you must choose one of the available transport-layer protocols. How do you make this choice? Most likely, you would study the services provided by the available transport-layer protocols, and then pick the protocol with the services that best match your application's needs.

What are the services that a transport-layer protocol can offer to applications invoking it? We can broadly classify the possible services along four dimensions: reliable data transfer, throughput, timing, and security.

Reliable Data Transfer

As discussed in Chapter 1, packets can get lost within a computer network.
For example, a packet can overflow a buffer in a router, or can be discarded by a host or router after having some of its bits corrupted.

For many applications—such as electronic mail, file transfer, remote host access, Web document transfers, and financial applications—data loss can have devastating consequences (in the latter case, for either the bank or the customer!).
Thus, to support these applications, something has to be done to guarantee that the data sent by one end of the application is delivered correctly and completely to the other end of the application.
If a protocol provides such a guaranteed data delivery service, it is said to provide **reliable data transfer**.

One important service that a transport-layer protocol can potentially provide to an application is process-to-process reliable data transfer.
When a transport protocol provides this service, the sending process can just pass its data into the socket and know with complete confidence that the data will arrive without errors at the receiving process.

When a transport-layer protocol doesn't provide reliable data transfer, some of the data sent by the sending process may never arrive at the receiving process.
This may be acceptable for **loss-tolerant applications**, most notably multimedia applications such as conversational audio/video that can tolerate some amount of data loss.

Throughput

In Chapter 1, we introduced the concept of available throughput, which, in the context of a communication session between two processes along a network path, is the rate at which the sending process can deliver bits to the receiving process.

Because other sessions will be sharing the bandwidth along the network path, and because these other sessions will be coming and going, the available throughput can fluctuate with time.

These observations lead to another natural service that a transport-layer protocol could provide, namely, guaranteed available throughput at some specified rate.

With such a service, the application could request a guaranteed throughput of r bits/sec, and the transport protocol would then ensure that the available throughput is always at least r bits/sec.

Such a guaranteed throughput service would appeal to many applications.

For example, if an Internet telephony application encodes voice at 32 kbps, it needs to send data into the network and have data delivered to the receiving application at this rate.
If the transport protocol cannot provide this throughput, the application would need to encode at a lower rate (and receive enough throughput to sustain this lower coding rate) or may have to give up, since receiving, say, half of the needed throughput is of little or no use to this Internet telephony application.

Applications that have throughput requirements are said to be **bandwidth-sensitive applications**. Many current multimedia applications are bandwidth sensitive, although some multimedia applications may use adaptive coding techniques to encode digitized voice or video at a rate that matches the currently available throughput.

While bandwidth-sensitive applications have specific throughput requirements, **elastic applications** can make use of as much, or as little, throughput as happens to be available. Electronic mail, file transfer, and Web transfers are all elastic applications.
Of course, the more throughput, the better.

Timing

A transport-layer protocol can also provide timing guarantees.

As with throughput guarantees, timing guarantees can come in many shapes and forms.
An example guarantee might be that every bit that the sender pumps into the socket arrives at the receiver's socket no more than 100 msec later.
Such a service would be appealing to interactive real-time applications, such as Internet telephony, virtual environments, teleconferencing, and multiplayer games, all of which require tight timing constraints on data delivery in order to be effective, see [Gauthier 1999; Ramjee 1994].

Long delays in Internet telephony, for example, tend to result in unnatural pauses in the conversation; in a multiplayer game or virtual interactive environment, a long delay between taking an action and seeing the response from the environment (for example, from another player at the end of an end-to-end connection) makes the application feel less realistic.

For non-real-time applications, lower delay is always preferable to higher delay, but no tight constraint is placed on the end-to-end delays.

Security

Finally, a transport protocol can provide an application with one or more security services.

For example, in the sending host, a transport protocol can encrypt all data transmitted by the sending process, and in the receiving host, the transport-layer protocol can decrypt the data before delivering the data to the receiving process.
Such a service would provide confidentiality between the two processes, even if the data is somehow observed between sending and receiving processes.

A transport protocol can also provide other security services in addition to confidentiality, including data integrity and end-point authentication, topics that we'll cover in detail in Chapter 8.

2.1.4 Transport Services Provided by the Internet

Up until this point, we have been considering transport services that a computer network *could* provide in general.

Let's now get more specific and examine the type of transport services provided by the Internet.
The Internet (and, more generally, TCP/IP networks) makes two transport protocols available to applications, UDP and TCP.

When you (as an application developer) create a new network application for the Internet, one of the first decisions you have to make is whether to use UDP or TCP.
Each of these protocols offers a different set of services to the invoking applications.

Figure 2.4 shows the service requirements for some selected applications.

Application	Data Loss	Throughput	Time-Sensitive
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony/ Video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

Figure 2.4 ♦ Requirements of selected network applications

TCP Services

The TCP service model includes a connection-oriented service and a reliable data transfer service.

When an application invokes TCP as its transport protocol, the application receives both of these services from TCP.

- *Connection-oriented service.*

TCP has the client and server exchange transport-layer control information with each other *before* the application-level messages begin to flow.

This so-called handshaking procedure alerts the client and server, allowing them to prepare for an onslaught of packets.

After the handshaking phase, a **TCP connection** is said to exist between the sockets of the two processes.

The connection is a full-duplex connection in that the two processes can send messages to each other over the connection at the same time.

When the application finishes sending messages, it must tear down the connection.

In Chapter 3, we'll discuss connection-oriented service in detail and examine how it is implemented.

- *Reliable data transfer service.*

The communicating processes can rely on TCP to deliver all data sent without error and in the proper order.
When one side of the application passes a stream of bytes into a socket, it can count on TCP to deliver the same stream of bytes to the receiving socket, with no missing or duplicate bytes.

TCP also includes a congestion-control mechanism, a service for the general welfare of the Internet rather than for the direct benefit of the communicating processes.
The TCP congestion-control mechanism throttles a sending process (client or server) when the network is congested between sender and receiver.

UDP Services

UDP is a no-frills, lightweight transport protocol, providing minimal services.

UDP is connectionless, so there is no handshaking before the two processes start to communicate.
UDP provides an unreliable data transfer service—that is, when a process sends a message into a UDP socket, UDP provides *no* guarantee that the message will ever reach the receiving process. Furthermore, messages that do arrive at the receiving process may arrive out of order.

UDP does not include a congestion-control mechanism, so the sending side of UDP can pump data into the layer below (the network layer) at any rate it pleases.

Services Not Provided by Internet Transport Protocols

We have organized transport protocol services along four dimensions: reliable data transfer, throughput, timing, and security.
Which of these services are provided by TCP and UDP? We have already noted that TCP provides reliable end-to-end data transfer.
And we also know that TCP can be easily enhanced at the application layer with TLS to provide security services.

But in our brief description of TCP and UDP, conspicuously missing was any mention of throughput or timing guarantees—services *not* provided by today's Internet transport protocols.
Does this mean that time-sensitive applications such as Internet telephony cannot run in today's Internet? The answer is clearly no—the Internet has been hosting time-sensitive applications for many years.
These applications often work fairly well because they have been designed to cope, to the greatest extent possible, with this lack of guarantee.

In summary, today's Internet can often provide satisfactory service to time-sensitive applications, but it cannot provide any timing or throughput guarantees.

Figure 2.5 indicates the transport protocols used by some popular Internet applications.
We see that e-mail, remote terminal access, the Web, and file transfer all use TCP.
These applications have chosen TCP primarily because TCP provides reliable data transfer, guaranteeing that all data will eventually get to its destination.

Because Internet telephony applications (such as Skype) can often tolerate some loss but require a minimal rate to be effective, developers of Internet telephony applications usually prefer to run their applications over UDP, thereby circumventing TCP's congestion control mechanism and packet overheads.
But because many firewalls are configured to block (most types of) UDP traffic, Internet telephony applications often are designed to use TCP as a backup if UDP communication fails.

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g., YouTube), DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary (e.g., Skype)	UDP or TCP

Figure 2.5 ♦ Popular Internet applications, their application-layer

2.1.5 Application-Layer Protocols

We have just learned that network processes communicate with each other by sending messages into sockets.
But how are these messages structured? What are the meanings of the various fields in the messages? When do the processes send the messages? These questions bring us into the realm of application-layer protocols.

An **application-layer protocol** defines how an application's processes, running on different end systems, pass messages to each other.

In particular, an application-layer protocol defines:

- The types of messages exchanged, for example, request messages and response messages
- The syntax of the various message types, such as the fields in the message and how the fields are delineated
- The semantics of the fields, that is, the meaning of the information in the fields
- Rules for determining when and how a process sends messages and responds to messages

Some application-layer protocols are specified in RFCs and are therefore in the public domain.
For example, the Web's application-layer protocol, HTTP (the Hyper Text Transfer Protocol [RFC 7230]), is available as an RFC.
If a browser developer follows the rules of the HTTP RFC, the browser will be able to retrieve Web pages from any Web server that has also followed the rules of the HTTP RFC.

Many other application-layer protocols are proprietary and intentionally not available in the public domain.
For example, Skype uses proprietary application-layer protocols.

It is important to distinguish between network applications and application-layer protocols. An application-layer protocol is only one piece of a network application (albeit, a very important piece of the application from our point of view!).

Let’s look at a couple of examples.
The Web is a client-server application that allows users to obtain documents from Web servers on demand.
The Web application consists of many components, including a standard for document formats (that is, HTML), Web browsers (for example, Chrome and Microsoft Internet Explorer), Web servers (for example, Apache and Microsoft servers), and an application-layer protocol.

The Web’s application-layer protocol, HTTP, defines the format and sequence of messages exchanged between browser and Web server.
Thus, HTTP is only one piece (albeit, an important piece) of the Web application.

As another example, Netflix’s video service also has many components, including servers that store and transmit videos, other servers that manage billing and other client functions, clients (e.g., the Netflix app on your smartphone, tablet, or computer), and an application-level DASH protocol defines the format and sequence of messages exchanged between a Netflix server and client.
Thus, DASH is only one piece (albeit, an important piece) of the Netflix application.

2.1.6 Network Applications Covered in This Book

New applications are being developed every day.
Rather than covering a large number of Internet applications in an encyclopedic manner, we have chosen to focus on a small number of applications that are both pervasive and important.

In this chapter, we discuss five important applications: the Web, electronic mail, directory service, video streaming, and P2P applications.

We first discuss the Web, not only because it is an enormously popular application, but also because its application-layer protocol, HTTP, is straightforward and easy to understand.

We then discuss electronic mail, the Internet's first killer application.
E-mail is more complex than the Web in the sense that it makes use of not one but several application-layer protocols.

After e-mail, we cover DNS, which provides a directory service for the Internet.
Most users do not interact with DNS directly; instead, users invoke DNS indirectly through other applications (including the Web, file transfer, and electronic mail).
DNS illustrates nicely how a piece of core network functionality (network-name to network-address translation) can be implemented at the application layer in the Internet.

We then discuss P2P file sharing applications, and complete our application study by discussing video streaming on demand, including distributing stored video over content distribution networks.

2.2 The Web and HTTP

Until the early 1990s, the Internet was used primarily by researchers, academics, and university students to log in to remote hosts, to transfer files from local hosts to remote hosts and vice versa, to receive and send news, and to receive and send electronic mail.
Although these applications were (and continue to be) extremely useful, the Internet was essentially unknown outside of the academic and research communities.

Then, in the early 1990s, a major new application arrived on the scene—the World Wide Web [Berners-Lee 1994].
The Web was the first Internet application that caught the general public's eye.
It dramatically changed how people interact inside and outside their work environments.
It elevated the Internet from just one of many data networks to essentially the one and only data network.
Perhaps what appeals the most to users is that the Web operates *on demand*.
Users receive what they want, when they want it.
This is unlike traditional broadcast radio and television, which force users to tune in when the content provider makes the content available.

In addition to being available on demand, the Web has many other wonderful features that people love and cherish.
It is enormously easy for any individual to make information available over the Web—everyone can become a publisher at extremely low cost.
Hyperlinks and search engines help us navigate through an ocean of information.
Photos and videos stimulate our senses.
Forms, JavaScript, video, and many other devices enable us to interact with pages and sites.

And the Web and its protocols serve as a platform for YouTube, Web-based e-mail (such as Gmail), and most mobile Internet applications, including Instagram and Google Maps.

2.2.1 Overview of HTTP

The **Hyper Text Transfer Protocol (HTTP)**, the Web's application-layer protocol, is at the heart of the Web.
It is defined in [RFC 1945], [RFC 7230] and [RFC 7540].

HTTP is implemented in two programs: a client program and a server program.
The client program and server program, executing on different end systems, talk to each other by exchanging HTTP messages.
HTTP defines the structure of these messages and how the client and server exchange the messages.

Before explaining HTTP in detail, we should review some Web terminology.
A **Web page** (also called a document) consists of objects.

An **object** is simply a file—such as an HTML file, a JPEG image, a Javascrpt file, a CCS style sheet file, or a video clip—that is addressable by a single URL.

Most Web pages consist of a **base HTML file** and several referenced objects.
For example, if a Web page contains HTML text and five JPEG images, then the Web page has six objects: the base HTML file plus the five images.
The base HTML file references the other objects in the page with the objects' URLs.

Each URL has two components: the hostname of the server that houses the object and the object's path name.
For example, the URL
`http://www.someSchool.edu/someDepartment/picture.gif` has
`www.someSchool.edu` for a hostname and
`/someDepartment/picture.gif` for a path name.

Because **Web browsers** (such as Internet Explorer and Chrome) implement the client side of HTTP, in the context of the Web, we will use the words *browser* and *client* interchangeably.

Web servers, which implement the server side of HTTP, house Web objects, each addressable by a URL.
Popular Web servers include Apache and Microsoft Internet Information Server.

HTTP defines how Web clients request Web pages from Web servers and how servers transfer Web pages to clients.

We discuss the interaction between client and server in detail later, but the general idea is illustrated in Figure 2.6.

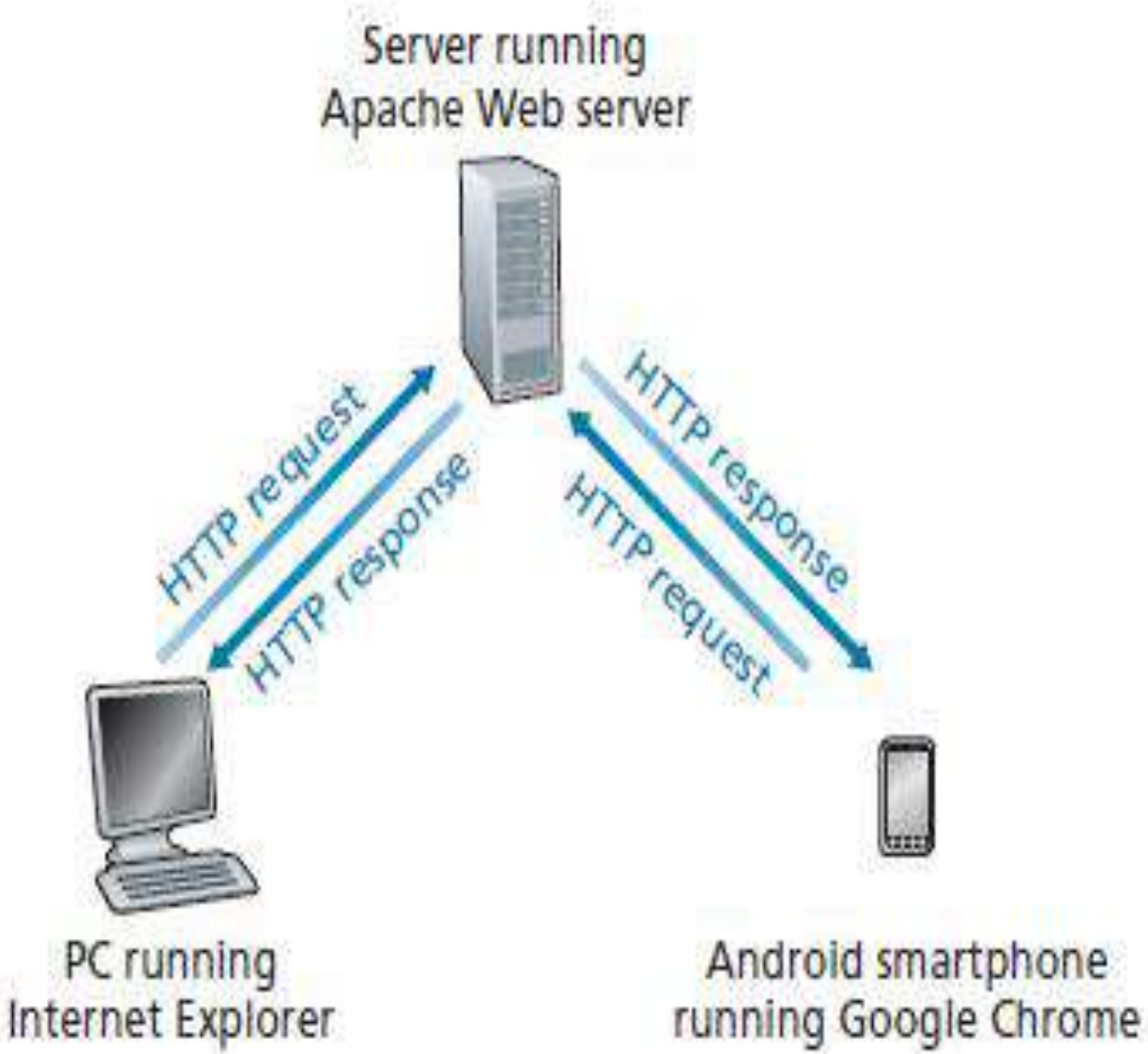


Figure 2.6 • HTTP request-response behavior

When a user requests a Web page (for example, clicks on a hyperlink), the browser sends HTTP request messages for the objects in the page to the server.
The server receives the requests and responds with HTTP response messages that contain the objects.

HTTP uses TCP as its underlying transport protocol (rather than running on top of UDP).

The HTTP client first initiates a TCP connection with the server.
Once the connection is established, the browser and the server processes access TCP through their socket interfaces.
As described in Section 2.1, on the client side the socket interface is the door between the client process and the TCP connection; on the server side it is the door between the server process and the TCP connection.
The client sends HTTP request messages into its socket interface and receives HTTP response messages from its socket interface.

Similarly, the HTTP server receives request messages from its socket interface and sends response messages into its socket interface.

Once the client sends a message into its socket interface, the message is out of the client's hands and is "in the hands" of TCP.

Recall from Section 2.1 that TCP provides a reliable data transfer service to HTTP.
This implies that each HTTP request message sent by a client process eventually arrives intact at the server; similarly, each HTTP response message sent by the server process eventually arrives intact at the client.

Here we see one of the great advantages of a layered architecture—HTTP need not worry about lost data or the details of how TCP recovers from loss or reordering of data within the network.
That is the job of TCP and the protocols in the lower layers of the protocol stack.

It is important to note that the server sends requested files to clients without storing any state information about the client.
If a particular client asks for the same object twice in a period of a few seconds, the server does not respond by saying that it just served the object to the client; instead, the server resends the object, as it has completely forgotten what it did earlier.

Because an HTTP server maintains no information about the clients, HTTP is said to be a **stateless protocol**.

We also remark that the Web uses the client-server application architecture, as described in Section 2.1.
A Web server is always on, with a fixed IP address, and it services requests from potentially millions of different browsers.

The original version of HTTP is called HTTP/1.0 and dates back to the early 1990's [RFC 1945].
As of 2020, the majority of HTTP transactions take place over HTTP/1.1 [RFC 7230].

However, increasingly browsers and Web servers also support a new version of HTTP called HTTP/2 [RFC 7540].
At the end of this section, we'll provide an introduction to HTTP/2.

2.2.2 Non-Persistent and Persistent Connections

In many Internet applications, the client and server communicate for an extended period of time, with the client making a series of requests and the server responding to each of the requests.

When this client-server interaction is taking place over TCP, the application developer needs to make an important decision—should each request/response pair be sent over a *separate* TCP connection, or should all of the requests and their corresponding responses be sent over the *same* TCP connection? In the former approach, the application is said to use **non-persistent connections**; and in the latter approach, **persistent connections**.

To gain a deep understanding of this design issue, let's examine the advantages and disadvantages of persistent connections in the context of a specific application, namely, HTTP, which can use both non-persistent connections and persistent connections.

Although HTTP uses persistent connections in its default mode, HTTP clients and servers can be configured to use non-persistent connections instead.

HTTP with Non-Persistent Connections

Let's walk through the steps of transferring a Web page from server to client for the case of non-persistent connections.

Let's suppose the page consists of a base HTML file and 10 JPEG images, and that all 11 of these objects reside on the same server.

Further suppose the URL for the base HTML file is

`http://www.someSchool.edu/someDepartment/home.index`

Here is what happens:

1. The HTTP client process initiates a TCP connection to the server `www.someSchool.edu` on port number 80, which is the default port number for HTTP. Associated with the TCP connection, there will be a socket at the client and a socket at the server.

2. The HTTP client sends an HTTP request message to the server via its socket. The request message includes the path name `/someDepartment/home.index`. (We will discuss HTTP messages in some detail below.)

3. The HTTP server process receives the request message via its socket, retrieves the object `/someDepartment/home.index` from its storage (RAM or disk), encapsulates the object in an HTTP response message, and sends the response message to the client via its socket.

4. The HTTP server process tells TCP to close the TCP connection.
(But TCP doesn't actually terminate the connection until it knows for sure that the client has received the response message intact.)

5. The HTTP client receives the response message.
The TCP connection terminates.
The message indicates that the encapsulated object is an HTML file.
The client extracts the file from the response message, examines the HTML file, and finds references to the 10 JPEG objects.

6. The first four steps are then repeated for each of the referenced JPEG objects.

As the browser receives the Web page, it displays the page to the user.

Two different browsers may interpret (that is, display to the user) a Web page in somewhat different ways.
HTTP has nothing to do with how a Web page is interpreted by a client.

The HTTP specifications ([RFC 1945] and [RFC 7540]) define only the communication protocol between the client HTTP program and the server HTTP program.

The steps above illustrate the use of non-persistent connections, where each TCP connection is closed after the server sends the object—the connection does not persist for other objects.

Note that each non-persistent TCP connection transports exactly one request message and one response message.
Thus, in this example, when a user requests the Web page, 11 TCP connections are generated.
In the steps described above, we were intentionally vague about whether the client obtains the 10 JPEGs over 10 serial TCP connections, or whether some of the JPEGs are obtained over parallel TCP connections.

Indeed, users can configure some browsers to control the degree of parallelism.
Browsers may open multiple TCP connections and request different parts of the Web page over the multiple connections.

As we'll see in the next chapter, the use of parallel connections shortens the response time.

Before continuing, let's do a back-of-the-envelope calculation to estimate the amount of time that elapses from when a client requests the base HTML file until the entire file is received by the client.

To this end, we define the **round-trip time (RTT)**, which is the time it takes for a small packet to travel from client to server and then back to the client.
The RTT includes packet-propagation delays, packet-queuing delays in intermediate routers and switches, and packet-processing delays.
(These delays were discussed in Section 1.4.)

Now consider what happens when a user clicks on a hyperlink.

As shown in Figure 2.7, this causes the browser to initiate a TCP connection between the browser and the Web server; this involves a "three-way handshake"—the client sends a small TCP segment to the server, the server acknowledges and responds with a small TCP segment, and, finally, the client acknowledges back to the server.

The first two parts of the three-way handshake take one RTT.

After completing the first two parts of the handshake, the client sends the HTTP request message combined with the third part of the three-way handshake (the acknowledgment) into the TCP connection.

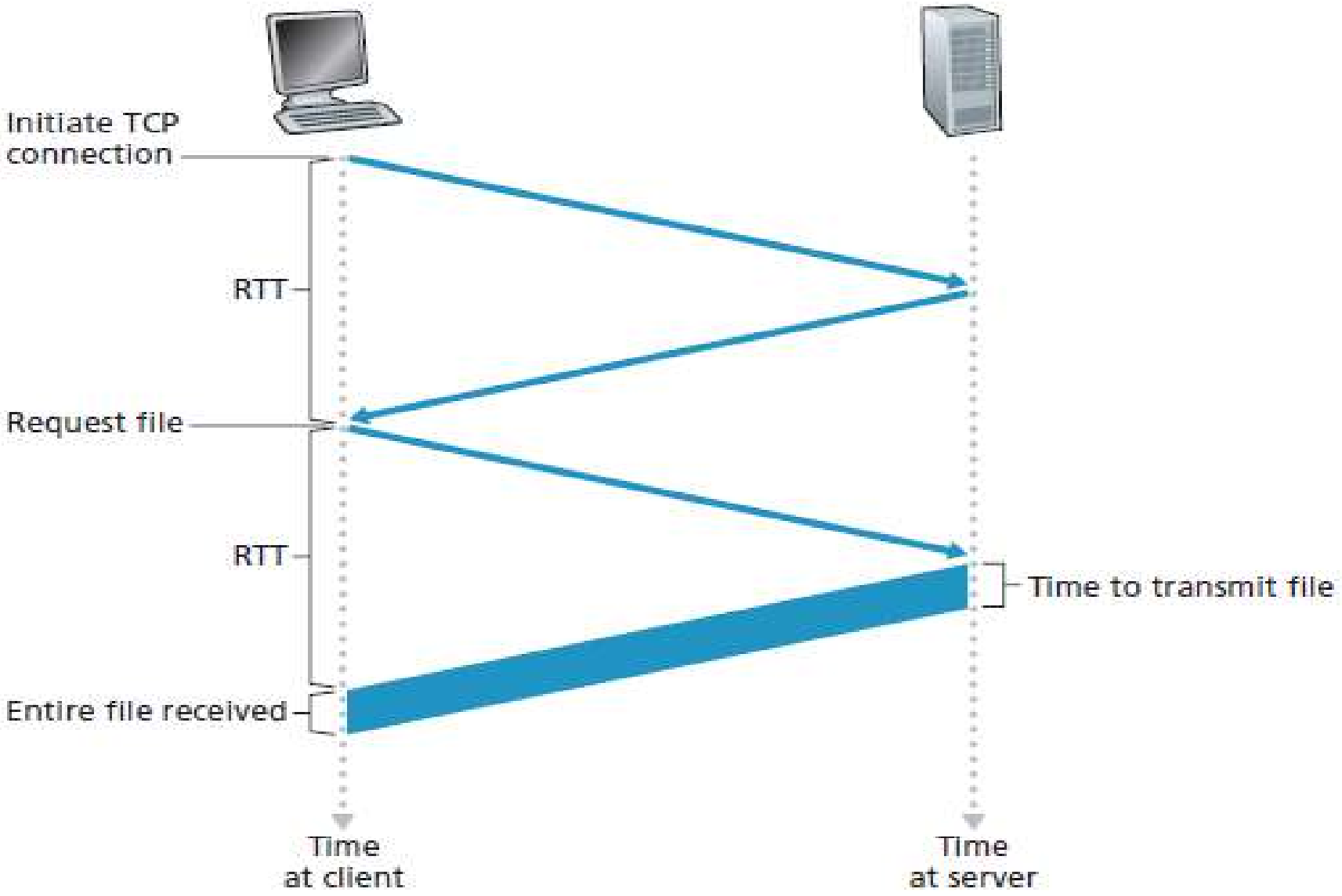


Figure 2.7 • Back-of-the-envelope calculation for the time needed to request and receive an HTML file

The first two parts of the three-way handshake take one RTT.

After completing the first two parts of the handshake, the client sends the HTTP request message combined with the third part of the three-way handshake (the acknowledgment) into the TCP connection.

Once the request message arrives at the server, the server sends the HTML file into the TCP connection.
This HTTP request/response eats up another RTT.

Thus, roughly, the total response time is two RTTs plus the transmission time at the server of the HTML file.

HTTP with Persistent Connections

Non-persistent connections have some shortcomings.

First, a brand-new connection must be established and maintained for *each requested object*. For each of these connections, TCP buffers must be allocated and TCP variables must be kept in both the client and server.
This can place a significant burden on the Web server, which may be serving requests from hundreds of different clients simultaneously.

Second, as we just described, each object suffers a delivery delay of two RTTs—one RTT to establish the TCP connection and one RTT to request and receive an object.

With HTTP/1.1 persistent connections, the server leaves the TCP connection open after sending a response.
Subsequent requests and responses between the same client and server can be sent over the same connection.

In particular, an entire Web page (in the example above, the base HTML file and the 10 images) can be sent over a single persistent TCP connection.
Moreover, multiple Web pages residing on the same server can be sent from the server to the same client over a single persistent TCP connection.

These requests for objects can be made back-to-back, without waiting for replies to pending requests (pipelining).
Typically, the HTTP server closes a connection when it isn't used for a certain time (a configurable timeout interval).
When the server receives the back-to-back requests, it sends the objects back-to-back.

The default mode of HTTP uses persistent connections with pipelining.

2.2.3 HTTP Message Format

The HTTP specifications [RFC 1945; RFC 7230; RFC 7540] include the definitions of the HTTP message formats.

There are two types of HTTP messages, request messages and response messages, both of which are discussed below.

HTTP Request Message

Below we provide a typical HTTP request message:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

We can learn a lot by taking a close look at this simple request message.

First of all, we see that the message is written in ordinary ASCII text, so that your ordinary computer-literate human being can read it.

Second, we see that the message consists of five lines, each followed by a carriage return and a line feed.
The last line is followed by an additional carriage return and line feed.
Although this particular request message has five lines, a request message can have many more lines or as few as one line.

The first line of an HTTP request message is called the **request line**; the subsequent lines are called the **header lines**.

The request line has three fields: the method field, the URL field, and the HTTP version field.

The method field can take on several different values, including `GET`, `POST`, `HEAD`, `PUT`, and `DELETE`.

The great majority of HTTP request messages use the `GET` method.
The `GET` method is used when the browser requests an object, with the requested object identified in the URL field.
In this example, the browser is requesting the object `/somedir/page.html`.

The version is self-explanatory; in this example, the browser implements version HTTP/1.1.

Now let's look at the header lines in the example.

The header line `Host:www.someschool.edu` specifies the host on which the object resides.
You might think that this header line is unnecessary, as there is already a TCP connection in place to the host.
But, as we'll see in Section 2.2.5, the information provided by the host header line is required by Web proxy caches.

By including the `Connection: close` header line, the browser is telling the server that it doesn't want to bother with persistent connections; it wants the server to close the connection after sending the requested object.

The `User-agent:` header line specifies the user agent, that is, the browser type that is making the request to the server.
Here the user agent is Mozilla/5.0, a Firefox browser.
This header line is useful because the server can actually send different versions of the same object to different types of user agents.

Finally, the `Accept-language:` header indicates that the user prefers to receive a French version of the object, if such an object exists on the server; otherwise, the server should send its default version.
The `Accept-language:` header is just one of many content negotiation headers available in HTTP.

Having looked at an example, let's now look at the general format of a request message, as shown in Figure 2.8.
We see that the general format closely follows our earlier example.

You may have noticed, however, that after the header lines (and the additional carriage return and line feed) there is an "entity body."
The entity body is empty with the `GET` method, but is used with the `POST` method.

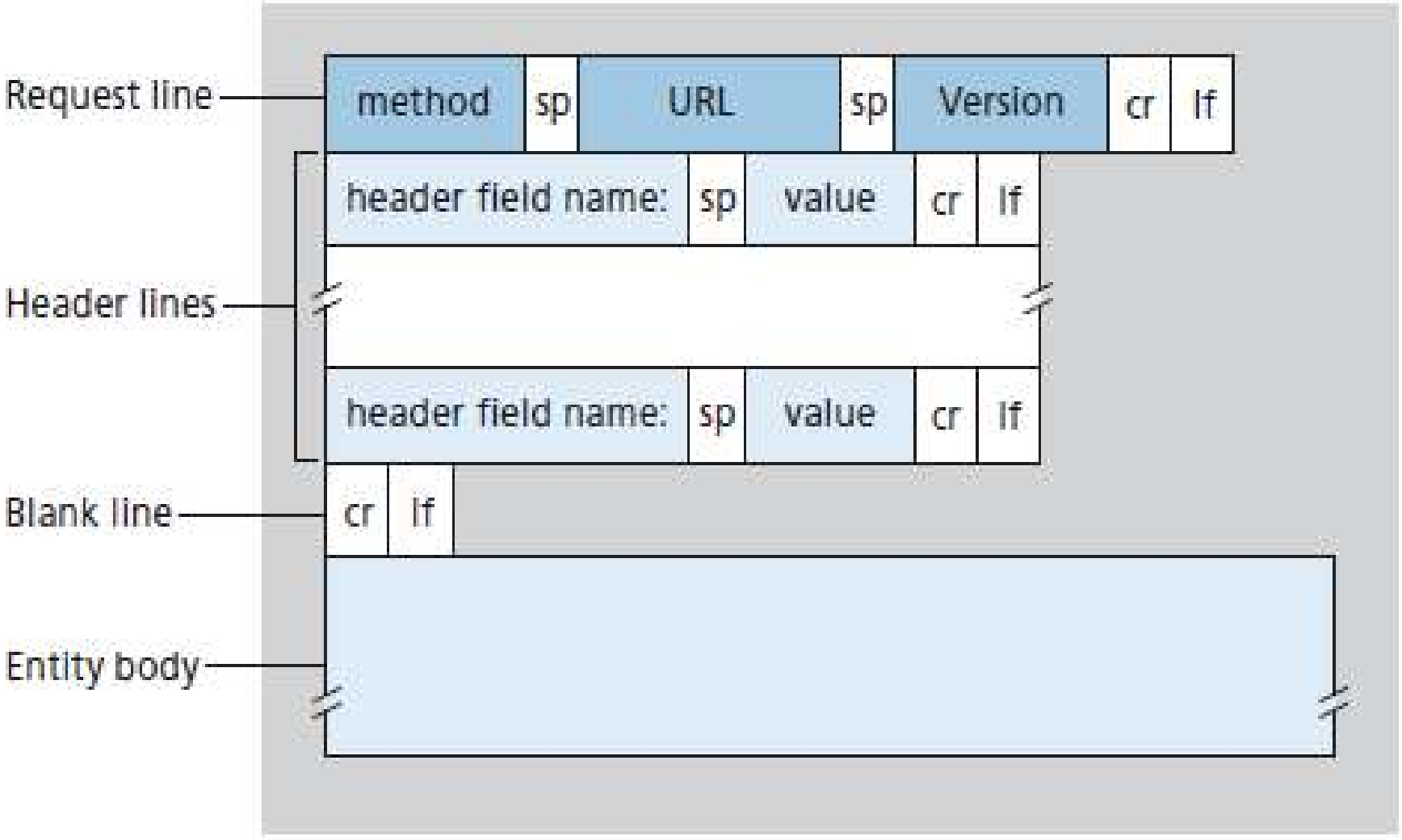


Figure 2.8 ♦ General format of an HTTP request message

An HTTP client often uses the `POST` method when the user fills out a form—for example, when a user provides search words to a search engine.
With a `POST` message, the user is still requesting a Web page from the server, but the specific contents of the Web page depend on what the user entered into the form fields.
If the value of the method field is `POST`, then the entity body contains what the user entered into the form fields.

We would be remiss if we didn't mention that a request generated with a form does not necessarily have to use the `POST` method.
Instead, HTML forms often use the `GET` method and include the inputted data (in the form fields) in the requested URL.
For example, if a form uses the `GET` method, has two fields, and the inputs to the two fields are `monkeys` and `bananas`, then the URL will have the structure `www.somesite.com/animalsearch?monkeys&bananas`.
In your day-today Web surfing, you have probably noticed extended URLs of this sort.

The `HEAD` method is similar to the `GET` method.
Application developers often use the `HEAD` method for debugging.

The `PUT` method is often used in conjunction with Web publishing tools.
It allows a user to upload an object to a specific path (directory) on a specific Web server.

The `DELETE` method allows a user, or an application, to delete an object on a Web server.

HTTP Response Message

Below we provide a typical HTTP response message. This response message could be the response to the example request message just discussed.

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03
GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

Let's take a careful look at this response message.
It has three sections: an initial **status line**, six **header lines**, and then the **entity body**.

The entity body is the meat of the message—it contains the requested object itself (represented by `data data data data data ...`).

The status line has three fields: the protocol version field, a status code, and a corresponding status message.
In this example, the status line indicates that the server is using HTTP/1.1 and that everything is OK (that is, the server has found, and is sending, the requested object).

Now let's look at the header lines.
The server uses the `Connection: close` header line to tell the client that it is going to close the TCP connection after sending the message.

The `Date:` header line indicates the time and date when the HTTP response was created and sent by the server.
Note that this is not the time when the object was created or last modified; it is the time when the server retrieves the object from its file system, inserts the object into the response message, and sends the response message.

The `Server:` header line indicates that the message was generated by an Apache Web server; it is analogous to the `User-agent:` header line in the HTTP request message.

The `Last-Modified:` header line indicates the time and date when the object was created or last modified.
The `Last-Modified:` header, which we will soon cover in more detail, is critical for object caching, both in the local client and in network cache servers (also known as proxy servers).

The `Content-Length:` header line indicates the number of bytes in the object being sent.

The `Content-Type:` header line indicates that the object in the entity body is HTML text.

Having looked at an example, let's now examine the general format of a response message, which is shown in Figure 2.9. This general format of the response message matches the previous example of a response message.

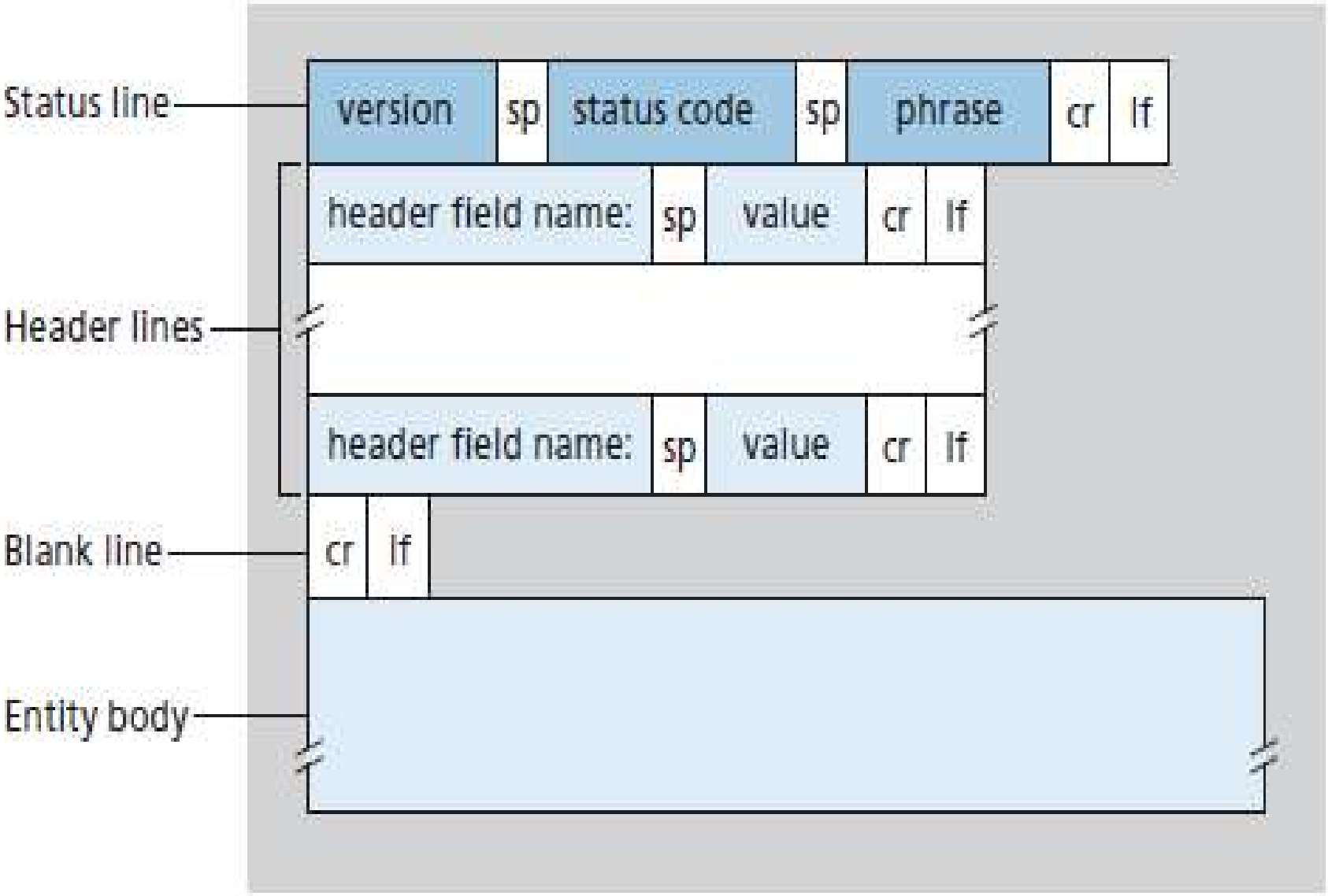


Figure 2.9 ♦ General format of an HTTP response message

Let's say a few additional words about status codes and their phrases.

The status code and associated phrase indicate the result of the request.
Some common status codes and associated phrases include:

- `200 OK:` Request succeeded and the information is returned in the response.

- `301 Moved Permanently:` Requested object has been permanently moved; the new URL is specified in `Location`: header of the response message.
The client software will automatically retrieve the new URL.

- `400 Bad Request:` This is a generic error code indicating that the request could not be understood by the server.

- `404 Not Found:` The requested document does not exist on this server.

- `505 HTTP Version Not Supported:` The requested HTTP protocol version is not supported by the server.

In this section, we discussed a number of header lines that can be used within HTTP request and response messages.
The HTTP specification defines many, many more header lines that can be inserted by browsers, Web servers, and network cache servers.
We have covered only a small number of the totality of header lines.

We'll cover a few more below and another small number when we discuss network Web caching in Section 2.2.5.